SISTEMAS OPERATIVOS

Aula Teórica 6

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

Problems

How one process can pass information to another?

Making sure two or more processes or threads do not get in each's way,
Ex. Two threads in an airline reservation system, each trying to grab the last seat on a plane for different customers.

Proper sequencing when dependencies are present,
Ex. if thread A produces data and thread B prints them, B as to wait until A has produced some more data before starting to print.

Solutions

1. Race Conditions
2. Critical Regions
3. Mutual Exclusion with Busy Waiting
4. Sleep and Wakeu
5. Semaphores
6. Mutexes
7. Monitors
8. Message Passing
9. Barriers
10. Priority Inversion
11. Avoiding Locks: Read-Copy-Update

1. Race Conditions

Example: Printer spooler

When a process wants to print a file, it enters the file name in a special **spooler directory**.

Another process the **printer deamon**, periodically checks to see if there are any files to be printed, and if there are, it prints them and remove their names from the directory.

If Process A and Process B, more or less simultaneously decide they want to queue a file for printing?

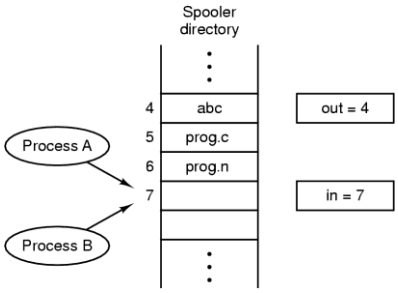
-Process A reads *in* and stores the value 7, in local variable *next_free_slot*.

-Just then a clock interrupt occurs and tghe CPU decides that process A has run long enough, so it switches to process B.

-Process B also reads *in* and also get 7. It too, stores its *in* into its local variable *next_free_slot*. At this moment both processes think the next availabe slot is 7. B continues running, store the name of its file in slot 7, and increment, *in*=next_free_slot+1=8.

- Eventualy process A runs again, starting from the place it left off, It looks to next_free_slot, finds 7, and stores its file name there, erasing the file name of process B just out there. Then compute , *in*=next_free_slot+1=8.

Process B will never receive any printer output.



Two processes want to access shared memory at same time.
shared variable *out* – point to the next file to be printed;
shared Variable *in* – points to the next free slot in the directory.

Situations like this, where two or more processes are reading and writing some shared data and the final result depends on who runs precisely when, are called **race conditions**.

With increasing parallelism due to increasing number of cores, race conditions are becoming more common.

2. Critical Regions (1)

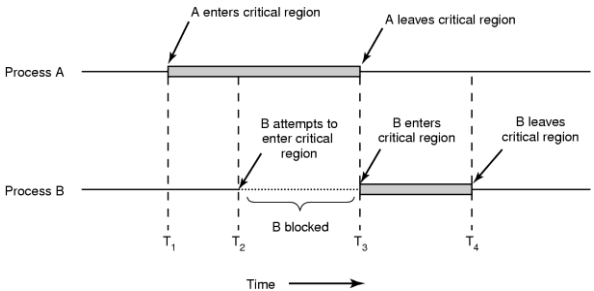
How do we avoid race conditions?

Mutual exclusion - Some way of making sure that if one process is using shared variable or file, the other processes will be excluded from doing the same thing.

Part of the time, a process is busy doing internal computations and other things that do not lead to race conditions. However, sometimes have to access shared memory or files, or do critical things that can lead to races.
The part of the program where the shared memory is accessed is called the **critical region** or **critical section**.

Four conditions to provide mutual exclusion

- No two processes simultaneaouly in critical region
- No assumptions made about speed or numbers of CPUs
- No process running outside its critical region may block another process
- No process must wait forever to enter its critical region



Mutual exclusion using critical regions

Proposals for achieving mutual exclusion

Disabling Interrupts

On a single-processor system the simplest solution is to have each process disable all interrupts just after entering its critical region and re-enable them just before leaving it. With interrupts disable no clock interrupts can occur. The CPU will not be switch to another process.

This approach is generally unattractive because it is unwise to give user processes the power to turn off interrupts. Furthermore, if the system is multiprocessor (with two or more CPU), disabling interrupts affects only the CPU that executed the disable instruction. The other ones will continue running and can access the shared memory.

Lock Variables

Consider a software solution, a shared *lock* variable. Where 0 means that no process is in its critical region, and 1 means that some process is in its critical region. Even with just two processes, in the proper operational conditions, it is quite easy to have both processes running in their critical region at the same time.

Strict Alternation

The integer variable *turn*, initially 0, keeps track of whose turn it is to enter the critical region and examine or update the shared memory.

-Process 0, inspects *turn* and finds it to be 0, and enter its critical region.
-Process 1 also finds it to be 0, and therefore sits in a tight loop continously testing turn to see when it becomes 1. Continously testing a variable until some value appears is called **busy waiting**.

It should usually be avoided, since it wastes CPU time.

```
while (TRUE) {
    while (turn != 0) { }      /* loop */
    critical_region();
    turn = 1;
    noncritical_region();
}
```
(a)

```
while (TRUE) {
    while (turn != 1) { }      /* loop */
    critical_region();
    turn = 0;
    noncritical_region();
}
```
(b)

Figure 2-23. A proposed solution to the critical-region problem. (a) Process 0. (b) Process 1. In both cases, be sure to note the semicolons terminating the while statements.

-When process 0 leaves critical region, it set *turn*=1, to allow process 1 to enter its critical region.

This will work, but taking turns is not a good idea when one of the processes is much slower than the other (violate the third condition to provide mutual exclusion).

3. Mutual Exclusion with Busy Waiting (3)

Peterson's Solution

Combining the idea of taking turns with the idea of lock variables and warning variables, a software solution with two procedures was presented.

Before using the shared variables (i.e. before entering its critical region), each process calls *enter_region* with its own process number, 0 or 1, as parameter.
This call will cause it to wait, if need be, until it is safe to enter.
After it has finished with the shared variables , the process calls *leave_region* to indicate it is done and allow the other process to enter , if it so desires.

```
#define FALSE  0
#define TRUE   1
#define N      2                        /* number of processes */

int turn;                               /* whose turn is it? */
int interested[N];                      /* all values initially 0 (FALSE) */

void enter_region(int process);         /* process is 0 or 1 */
{
     int other;                         /* number of the other process */

     other = 1 – process;               /* the opposite of process */
     interested[process] = TRUE;        /* show that you are interested */
     turn = process;                    /* set flag */
     while (turn == process && interested[other] == TRUE)  /* null statement */ ;
}

void leave_region(int process)          /* process: who is leaving */
{
     interested[process] = FALSE;       /* indicate departure from critical region */
}
```

Figure 2-24. Peterson's solution for achieving mutual exclusion.

Consider the case that both processes call *enter _region* almost simultaneously.
Both will store their process number in *turn*. Whichever store is done last is the one that counts; the first one is overwritten and lost. Suppose that process 1 store last, so *turn*=1. When both processes come to a while statement, process 0 executes it zero times and enters its critical region. Process 1 loops and does not enter its critical region until process 0 exits its critical region.

TSL Instruction (Test and Set Lock)

A proposal with a little help from the hardware.
Some computers , designed with multiple processors have an instruction like

TSL RX, LOCK

it reads the content of memory word *lock* into register RX and then stores a nonzero value at memory address *lock*. This operation are guaranteed to be indivisible – no other processor can access the memory word until the instruction is finished.

The CPU executing the TSL instruction **locks the memory bus** to prohibit other CPUs from accessing memory until it is done.

As in **all solutions based on critical region**, the process must call *enter_region* and *leave_region* **at the correct times** for the method to work. If one process cheats, the mutual exclusion will fail.

Alternative instrution to TSL is **XCGH** (in all Intel x86 CPUs) for low-level synchronization.

```
enter_region:
    TSL REGISTER,LOCK        | copy lock to register and set lock to 1
    CMP REGISTER,#0          | was lock zero?
    JNE enter_region         | if it was not zero, lock was set, so loop
    RET                      | return to caller; critical region entered

leave_region:
    MOVE LOCK,#0             | store a 0 in lock
    RET                      | return to caller
```

Figure 2-25. Entering and leaving a critical region using the TSL instruction.

```
enter_region:
    MOVE REGISTER,#1         | put a 1 in the register
    XCHG REGISTER,LOCK       | swap contents of register and lock variable
    CMP REGISTER,#0          | was lock zero?
    JNE enter_region         | if it was non zero, lock was set, so loop
    RET                      | return to caller; critical region entered

leave_region:
    MOVE LOCK,#0             | store a 0 in lock
    RET                      | return to caller
```

Figure 2-26. Entering and leaving a critical region using the XCHG instruction.

4. Sleep and Wakeup (1)

Both Peterson's solution and the solution using TSL or XCHG are correct, but both have the defect of requiring busy waiting.

Resuming these solutions,

> when a process wants to enter its critical region, it checks to see if the entry is allowed. If it is not, the process just sits in a tight loop burning the CPU while waiting until it is.

There are some interprocess communication primitives that block a process instead of wasting CPU time when they are not allowed to enter their critical regions.

The simplest: the pair **Sleep and Wakeup**

Sleep - is a system call that causes the caller to block (to be suspended until another process wakes it up);

Wakeup - is a system call with just one parameter, the process to be awakened.

PRODUCER – CONSUMER PROBLEM

Two processes share a common, **fixed-size buffer**, size N (the maximum number of items the buffer can hold).
-The Producer process, puts information into the buffer;
- The Consumer process, takes out information from the buffer.
Trouble comes:

-When the producer wants to put a new item in the buffer, but it is already full. The solution is for the producer to go to sleep, and be awakened when the consumer , takes it out.

-Similarly, if the consumer wants to remove a item from the buffer and sees that the buffer is empty, it goes to sleep until the producer puts something in the buffer and wakes it up.

In the figure there is an example, with variable *count* to keep track of the number of items in the buffer.

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          item = produce_item();                /* generate next item */
          if (count == N) sleep();              /* if buffer is full, go to sleep */
          insert_item(item);                    /* put item in buffer */
          count = count + 1;                    /* increment count of items in buffer */
          if (count == 1) wakeup(consumer);     /* was buffer empty? */
     }
}

void consumer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          if (count == 0) sleep();              /* if buffer is empty, got to sleep */
          item = remove_item();                 /* take item from buffer */
          count = count - 1;                    /* decrement count of items in buffer */
          if (count == N - 1) wakeup(producer); /* was buffer full? */
          consume_item(item);                   /* print item */
     }
}
```

Figure 2-27. The producer-consumer problem with a fatal race condition.

Race condition appears, as usual, with unconstrained access to shared variable *count*..., follow the example

-The buffer is empty and the consumer has just read *count* to see if it is 0. At that moment the scheduler decides to stop running the consumer temporarily and start running the producer.

-The producer inserts an item in the buffer, *count*=*count*+1, and calls *wakeup* to wake the consumer up.

-Unfortunately, the consumer is not yet logically asleep, so the wakeup signal is lost. When the consumer next runs, it will test the previously read value of *count*, find it to be 0, and go to sleep. Sooner or later the producer will fill up the buffer and also go to sleep. Both will sleep forever.

PRODUCER – CONSUMER PROBLEM

The essence of the problem here is that a wakeup sent to a process That is not (yet) sleeping is lost.

If it were not lost, everything would work.

A quick fix is to modify the rules, and add a **wakeup waiting bit**

-When a wakeup in sent to a process that is still awake, the bit is set **ON.**

-When the process tries to go to sleep, if wakeup bit is ON, it will be turned **OFF**, but the process will stay awake.

-The consumer **clear the wakeup waiting bit in every iterations** of the loop**.**

In this example it works, but with more processes, we need more bits, and the problem persists, because its principle is still there.

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          item = produce_item();                /* generate next item */
          if (count == N) sleep();              /* if buffer is full, go to sleep */
          insert_item(item);                    /* put item in buffer */
          count = count + 1;                    /* increment count of items in buffer */
          if (count == 1) wakeup(consumer);     /* was buffer empty? */
     }
}

void consumer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          if (count == 0) sleep();              /* if buffer is empty, got to sleep */
          item = remove_item();                 /* take item from buffer */
          count = count - 1;                    /* decrement count of items in buffer */
          if (count == N - 1) wakeup(producer); /* was buffer full? */
          consume_item(item);                   /* print item */
     }
}
```

Figure 2-27. The producer-consumer problem with a fatal race condition.

A new integer variable to count the number of wakeups saved for future use, was suggested by E. W. Dijkstra in 1965, called it **semaphore**.

Down operation on a semaphore (based on sleep)
Checks to see if the value is greater than 0.
If so, it **decrements the value**, and just continues.
If the value is 0, the process is put to sleep without completing the down for the moment.

Checking the value, changing it, and possibly going to sleep, are all done as a single, indivisible **atomic action**. It is guaranteed that once a semaphore operation has started, no other process can access the semaphore until the operation has completed or blocked.

UP operation on a semaphore (based on wakeup)
Increments the value of the semaphore addressed.
If one or more processes were sleeping on that semaphore, unable to complete an earlier Down operation, one of them is chosen by the system (ex. random) and is allowed to complete its Down.
After an Up on a semaphore with processes sleeping on it, the semaphore will still have the value of 0.
However, there will be one fewer process sleeping on it.

PRODUCER – CONSUMER PROBLEM

Up and Down must be implemente as system calls, with the operating system briefly disabling all interrupts while it is testing the semaphore, updating it, or putting a process to sleep.
No harm is done in disabling interrupts, for just a few instructions. The semaphore operation just take a few nanoseconds, whereas the producer or consumer might take arbitrarily long.

The solution for producer–consumer uses three semaphores:

full – for counting the number of slots that are full;
empty – for counting the number of slots that are empty;
mutex – to make sure the producer and consumer do not access the buffer at the same time.

Semaphores that are initialized to 1 and used by two or more processes to ensure that only one of them can enter its critical region are called **binary semaphores**. (ex: mutex)

If a process does a DOWN just before entering its critical region and an UP just after leaving it, **mutual exclusion are guaranteed**.

```
#define N 100                                   /* number of slots in the buffer */
typedef int semaphore;                          /* semaphores are a special kind of int */
semaphore mutex = 1;                            /* controls access to critical region */
semaphore empty = N;                            /* counts empty buffer slots */
semaphore full = 0;                             /* counts full buffer slots */

void producer(void)
{
    int item;

    while (TRUE) {                              /* TRUE is the constant 1 */
        item = produce_item();                  /* generate something to put in buffer */
        down(&empty);                           /* decrement empty count */
        down(&mutex);                           /* enter critical region */
        insert_item(item);                      /* put new item in buffer */
        up(&mutex);                             /* leave critical region */
        up(&full);                              /* increment count of full slots */
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {                              /* infinite loop */
        down(&full);                            /* decrement full count */
        down(&mutex);                           /* enter critical region */
        item = remove_item();                   /* take item from buffer */
        up(&mutex);                             /* leave critical region */
        up(&empty);                             /* increment count of empty slots */
        consume_item(item);                     /* do something with the item */
    }
}
```

Figure 2-28. The producer-consumer problem using semaphores.

PRODUCER – CONSUMER PROBLEM

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          item = produce_item();                /* generate next item */
          if (count == N) sleep();              /* if buffer is full, go to sleep */
          insert_item(item);                    /* put item in buffer */
          count = count + 1;                    /* increment count of items in buffer */
          if (count == 1) wakeup(consumer);     /* was buffer empty? */
     }
}

void consumer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          if (count == 0) sleep();              /* if buffer is empty, got to sleep */
          item = remove_item();                 /* take item from buffer */
          count = count - 1;                    /* decrement count of items in buffer */
          if (count == N - 1) wakeup(producer); /* was buffer full? */
          consume_item(item);                   /* print item */
     }
}
```

Figure 2-27. The producer-consumer problem with a fatal race condition.

semaphores

```
#define N 100                                   /* number of slots in the buffer */
typedef int semaphore;                          /* semaphores are a special kind of int */
semaphore mutex = 1;                            /* controls access to critical region */
semaphore empty = N;                            /* counts empty buffer slots */
semaphore full = 0;                             /* counts full buffer slots */

void producer(void)
{
     int item;

     while (TRUE) {                             /* TRUE is the constant 1 */
          item = produce_item();                /* generate something to put in buffer */
          down(&empty);                         /* decrement empty count */
          down(&mutex);                         /* enter critical region */
          insert_item(item);                    /* put new item in buffer */
          up(&mutex);                           /* leave critical region */
          up(&full);                            /* increment count of full slots */
     }
}

void consumer(void)
{
     int item;

     while (TRUE) {                             /* infinite loop */
          down(&full);                          /* decrement full count */
          down(&mutex);                         /* enter critical region */
          item = remove_item();                 /* take item from buffer */
          up(&mutex);                           /* leave critical region */
          up(&empty);                           /* increment count of empty slots */
          consume_item(item);                   /* do something with the item */
     }
}
```

Figure 2-28. The producer-consumer problem using semaphores.

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

